

GenAI Training Day 0

Python Foundations

- Python is the primary language for LLM & GenAI development
- Strong ecosystem for AI, data processing, tooling, and infra
- Simple syntax enables fast experimentation and iteration

Python Basics in GenAI

- Scripts power ingestion, evaluation, and agent workflows
- Clear entry points (main.py) and dynamic typing
- Clean, descriptive, GenAI-friendly variable naming

Data Types, Structures & Operators

- **Core types:** int, float, string, boolean
- **Operators** for logic, comparison, and debugging LLM flows
- **Lists** → prompts & message history
- **Tuples** → fixed configuration values
- **Dictionaries** → API payloads, responses, metadata
- **Sets** → deduplication

Control Flow & Functions

- **if / elif / else** for pipeline decisions
- **for / while loops** for batching, retries, ingestion
- **Functions** encapsulate reusable GenAI logic
- **Inputs:** prompts, configs | **Outputs:** responses, embeddings

Libraries & Tooling

- **NumPy** for vector math and embeddings
- **Pandas** for logs, evaluations, and datasets
- NumPy arrays outperform Python lists for numerical work
- **python-dotenv** for environment variables and secrets

Project & Environment Management

- **pip** for basic package installation
- **requirements.txt** as the modern project standard
- **.gitignore** protects secrets and keeps repos clean
- **Virtual environments** ensure isolation and stability
- **.env** for secure API keys and configuration

Modern Dependency Management

- **pyproject.toml** is legacy and hard to reproduce
- **uv** provides fast installs, dependency resolution, and lockfiles

Key Takeaways

- Python is the foundation of GenAI systems
- Clean structure and environments are critical
- Modern tooling enables scalable, production-ready AI